

CLICK-A-THON 2026 · SONY LIV · CLICKHOUSE CLOUD

Pulse

Foreground-only concurrent viewers.



Orbitrons · Click-a-thon 2026 · ClickHouse Cloud
Dashboard · API · LibreChat · ClickStack · Langfuse

The problem

SONY LIV · WHAT “CONCURRENT VIEWERS” MUST MEAN

7.0M

RAW EVENTS

148K

ACTIVE SEGMENTS

$\sim 3 \times 10^5$

DELTA ROWS

6.7M

HEARTBEATS

Ask

- Peak & average **foreground-active** concurrency
- Minute, hour, day grain — same filters on all three
- Filter by platform, country, content, app version, languages, player version
- Dynamic JSON `properties.*` dimensions from CSV extras
- Incremental / open-session updates without full rebuild
- Sub Seconds latencies for dashboards

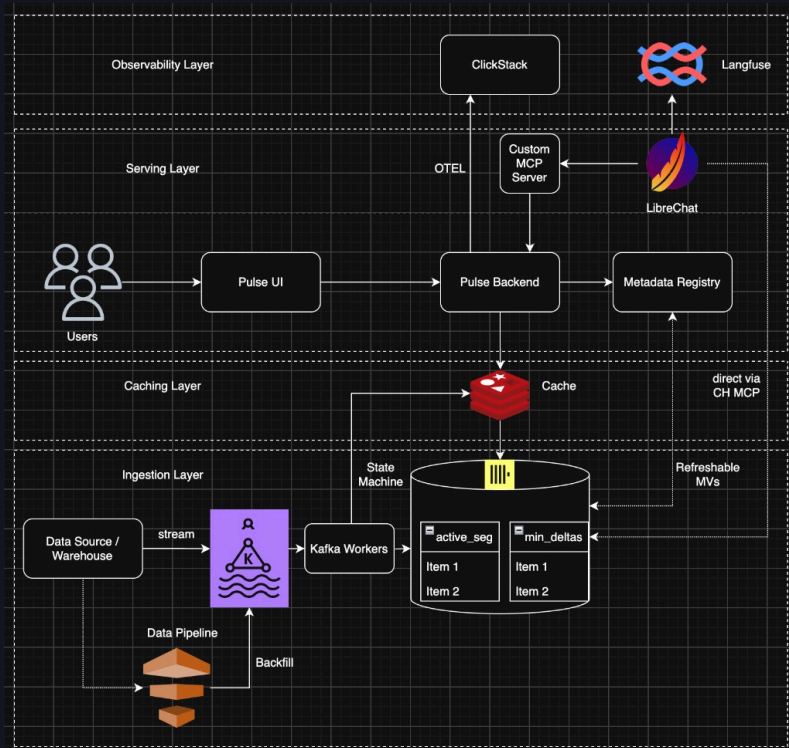
Foreground-State Logic

- App in foreground — not backgrounded
- Playback started — buffering counts; pause does *not*
- Session open — between start and end events
- Heartbeat liveness — stale sessions drop off

Pause markers live inside `VideoHeartbeat` via the `event` column — a classifier on `event_type` alone is blind to pause.

System HLD

FOUR LAYERS BOTTOM → TOP • SOLID = BUILT • DOTTED = DIRECT



01 INGESTION

KAFKA → WORKERS → CH

02 CACHING

PREFLIGHT + LIVE STATE

03 SERVING

API · UI · LIBRECHAT

04 OBSERVABILITY

CLICKSTACK · LANGFUSE

Layer 01 · Real-time ingestion

KAFKA WORKERS · REDIS HOT STATE · CLICKHOUSE SERVING LAYER

Live path

- Client playback events → **Kafka** → ingestion workers
- Per session: foreground-active state machine in **Redis** ($O(1)$ per event)
- Segment **closes** → compacted `±1` edges land in `minute_deltas`
- Segment **still open** → live row in `session_active_segments` (corrected at query time)
- Sub-second freshness — curve updates as viewers start/stop watching
- Same logic for CSV replay in the demo; prod swaps in a Kafka consumer

sub-second

horizontally scalable workers

State machine → sweep-line deltas

- Four rules: foreground · playing · not paused · session open
- Each active interval → one `+1` / `-1` pair (not per-minute explosion)
- Dimensions live on segments, not deltas. Hence new properties doesn't affect this table
- Dynamic JSON `properties` on segments — new Kafka fields without DDL churn

Redis holds **hot session state** for workers. ClickHouse is the durable store and what every consumer queries.

Layer 01 • Tables & correction

SERVING TABLES • COMPACTED HISTORY • LIVE OPEN SESSIONS

Serving tables

- `session_active_segments` — typed dims + JSON properties
- `minute_deltas` — narrow (minute, segment_id, ±1)
- `content_dict` • `properties_key_mappings` MV

① Compacted (historical)

- Finished intervals → merged ±1 in `minute_deltas`
- `SummingMergeTree` • opening balance + cumulative sum

Why two physical shapes

- **Closed** → compacted into `minute_deltas`
- **Open** → live rows in `session_active_segments`
- Workers emit deltas only on *close*

② Live correction

- Open sessions: `close_reason = ''` — not yet in deltas
- `open_edges` CTE synthesizes `+1/-1` from live rows
- `UNION ALL` with compacted deltas — one curve, filters on both

Layer 02 · Caching

REDIS — PREFLIGHT DEDUPE + LIVE SESSION STATE

Preflight — chart query cache

- Dashboard + bench hammer identical chart queries → singleflight dedupe
- Cache key = SHA256 query fingerprint; `SetNX` inflight lock
- Result JSON under `sony:ch:result:*` ; TTL configurable (default 1m)
- Redis down → graceful degrade to direct ClickHouse

singleflight

hot dashboard

Live state — streaming hot path

- `streamd` → per-session `Accumulator` in Redis
(`pulse:session:{id}`)
- Fixed TTL 72h from first seen; `pulse:active` set for O(1) count
- Same state machine as batch —
`TestStreamingMatchesBatch`
- `/live` API + `open_edges` fold open sessions into the curve

Two Redis roles, different semantics — preflight caches query *results*; livestate holds ingestion *state*.

Layer 03 · Query compiler

POST /API/V1/CONCURRENCY/CHART · ONE TEMPLATE, ALL GRAINS

Normative query shape

- Resolve filters → `segment_id IN (...)` semi-join
- Opening balance before window start
- Dense minute grid + cumulative `sum(delta)`
- **open_edges** — union live open sessions with compacted `minute_deltas`
- Bucket to hour/day from same minute series
- `dictGet(content_dict, ...)` for title / video_type

API surface

- `/api/v1/concurrency/chart` — peak, avg, timeseries, breakdown
- `/api/v1/concurrency/live` — Redis open sessions + CH curve
- `/api/v1/schema/*` — static + dynamic dimensions from MV
- React dashboard + live replay view
- Preflight wraps chart path; OTEL span per request

peak

avg

breakdown

properties.*

Layer 03 · Serving query & latencies

SAME COMPILED SQL FOR API · BENCH · DASHBOARD · 7M-EVENT UNSEEN DAY

30 ms

CH P50 SERVER-SIDE

49 ms

CH P90 SERVER-SIDE

~248 ms

E2E CHART (MINUTE)

243K

MAX ROWS READ

Compiled chart query

WITH

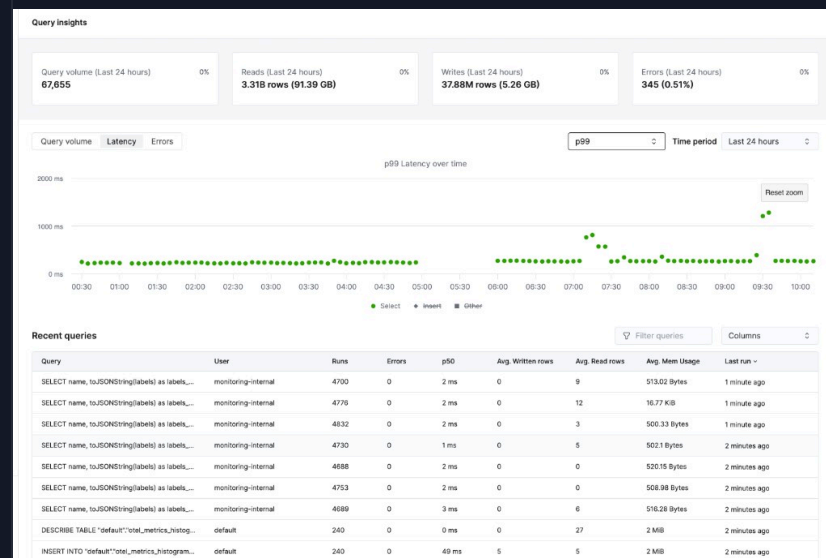
```
sel AS ( ... filtered segment ids ... ),
open_edges AS ( ... ±1 for still-open sessions ... ),
opening AS ( SELECT sum(delta) ... before window ... ),
net AS ( SELECT minute, sum(delta) ... in window ... ),
grid AS ( SELECT ... every minute in [start, end] ... ),
curve AS (
  SELECT g.minute,
    opening.c0 + sum(net) OVER (ORDER BY g.minute) AS concurrency
  FROM grid g LEFT JOIN net ...
)
```

SELECT minute, concurrency FROM curve ORDER BY minute;

minute_deltas

open_edges

no raw_events



Layer 03 · Dashboard & demo

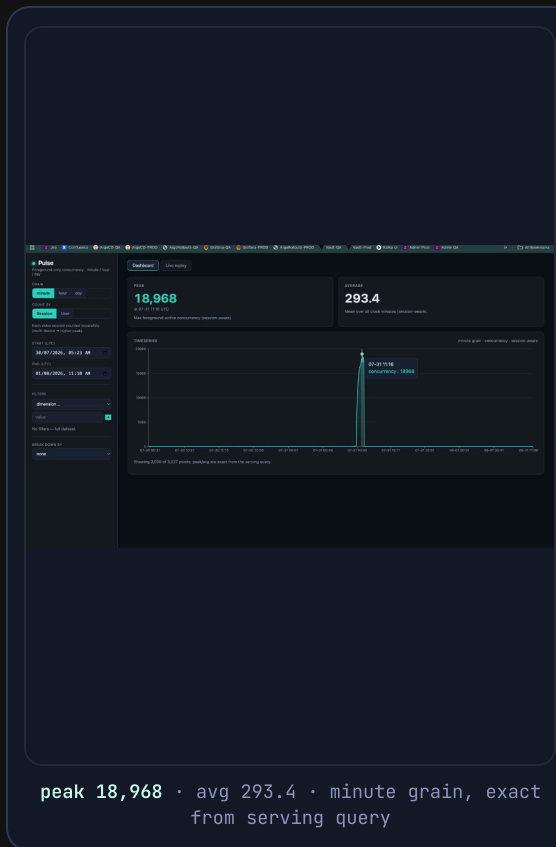
MINIMAL UI — MODEL AND SERVING LAYER ARE THE SCORE, NOT POLISH

React dashboard

- Peak / average cards + minute timeseries (Recharts)
- Grain toggle: minute · hour · day
- Dimension filters + dynamic `properties.*`
- Breakdown table — top-N by dimension

Live replay & incremental

- **Replay view** — animates the minute curve live
- **streamd** — CSV → Redis → CH on segment close
- `/api/v1/concurrency/live` — Redis gauge + CH union



Layer 03 · LibreChat + MCP

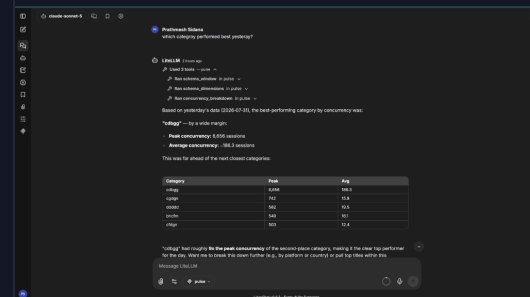
SECOND CONSUMER OF THE SERVING LAYER (DOTTED PATH ON HLD)

What we built

- LibreChat + official **ClickHouse MCP** (SSE)
- Read-only user `pulse_readonly` — no writes
- `system_prompt.md` — serving tables + semantics
- Compose profile `chat` — Mongo + MCP + LiteLLM

Honest limits

- **pulse MCP** proxies chart/breakdown — same compiler as the dashboard
- ClickHouse MCP: schema inspection only, not concurrency numbers
- Dynamic keys: `schema_dimensions` / `properties_key_mappings`



"Which category performed best yesterday?" → 3
pulse MCP tool calls

Layer 04 · Observability

CLICKSTACK (API / PIPELINE) · LANGFUSE (LLM TRACES)

ClickStack — OTLP → Cloud

- `clickstack-otel-collector` in docker-compose → `otel_*` tables
- Backend:
`OTEL_EXPORTER_OTLP_ENDPOINT=http://clickstack:4318`
- Spans: `concurrency.chart`, pipeline CLIs, service `pulse-concurrency-api`
- `smoke_integrations.sh` — API, CH, OTLP, MCP health

OpenTelemetry

internal/otelx

Langfuse — conversational plane

- LibreChat → LiteLLM proxy → upstream model
- `success_callback: ["langfuse"]` — prompts, tokens, MCP tool calls
- Keys in gitignored `.env` ; `verify_langfuse_traces.sh` smoke
- Complements ClickStack — does not replace API/pipeline tracing

HLD dotted LibreChat → Langfuse direct — actual path is **via LiteLLM proxy**.

Scaling to 100×

INTERPRETATION A — 100× SESSIONS IN THE SAME 12-DAY WINDOW (LIVE-EVENT DENSITY)

$\sim 3 \times 10^7$

DELTA ROWS

$\sim 1.9\text{M}$

SEGMENTS / DAY

<1 GB

SERVING LAYER

50–150 ms

FILTERED DAY QUERY

Why the model survives

- Deltas scale with **segments**, not minutes — no per-minute explosion
- Query output = one row per clock minute in window — invariant to session count
- Bounded semi-join + opening balance — $O(\text{window} + \text{lookback})$, not $O(\text{history})$
- Skip semi-join when no dimension filter — 3–8× on unfiltered queries
- Rollup `concurrency_minute_serving` when $p95 > 200$ ms (trigger in §15.5)

Triggers we watch

- Rollup `concurrency_minute_serving` when $p95 > 200$ ms
- Build batching when raw events $> 20\text{M}$ per cohort
- `REPLACE PARTITION` when `FINAL` $> 20\%$ of query time
- Numeric scaling triggers — measured, not guessed (see Assumptions slide)

Today every team's queries finish in ms — judges score **what we read** and this 100× reasoning, not stopwatch theatre.

Assumptions & limits

LOCKED SEMANTICS · ASSERTED PRECONDITIONS · OPEN QUESTIONS (Q1-Q5)

Lookback period (`MAX_SEGMENT_SPAN_HOURS = 72`)

- Query template bounds `sel` + opening balance to **window + 72h lookback**
- **Overlap bound** — theorem: segments outside the window contribute a cancelling ± 1 pair or nothing $\rightarrow$ answer-preserving
- **Lookback bound** — asserted precondition: valid only while no segment exceeds 72h
- **Impact at scale:** turns $O(\text{history})$ scans into $O(\text{window} + 72\text{h})$
- Same 72h bound reused for Redis TTL / streamed lateness window

Semantic parameters (`config.env`)

- **Pause excluded** — flip $\rightarrow \sim +2\%$ peak/avg (`evidence/sensitivity.md`)
- **Buffering included** — flip $\rightarrow \sim +34\%$ peak, $+42\%$ avg — dominant knob
- **Grace = 90s** — near-inert (0.87% of gaps $> 90\text{s}$); not the lever that fixes wrong answers
- **Any-overlap minute attribution** — highest-risk locked choice vs private ground truth
- **Average = all clock minutes** in window (including zeros) — narrow filters swing more if flipped
- **UTC bucketing** — day grain cuts at 00:00 UTC (05:30 IST); IST is a query-layer one-liner
- **Open segments** clamped to `least(last_hb + grace, watermark)` — no phantom tail past data end

Trust & evidence

WHY ANSWERS ARE DEFENSIBLE WITHOUT A PUBLISHED ANSWER KEY

Validation

- `cmd/validate` — automated invariants
- Delta vs minute-explosion arithmetic cross-check
- Hand-computed fixtures + sensitivity matrix
- `cmd/reconcile` — incremental correction tests

Benchmarks

- `cmd/bench` → `evidence/answers.json`
- Query log + parts read + ClickHouse Query Insights
- `unseen_day.sh` — full pipeline on held-out day
- Latency recorded; semantics scored first

Guardrails

- SQL grants: no `raw_events`, `readonly=1`
- Locked rules R1-R10 in `config.env`
- Same serving layer for API, bench, chat
- Sensitivity matrix — pause, heartbeat TTL, buffer policy
- Docs: presentation deck + `evidence/sensitivity.md`

Future scope

BUILT FOR HACKATHON CORRECTNESS · DESIGNED FOR PRODUCTION EVOLUTION

Ingestion & infra

- **Kafka consumer** — drop-in for CSV replay; same `Accumulator` + Redis hot path
- Redis cluster / stream partitioning for multi-worker ingest
- Build batching by session-start day — restartable cohorts at 90M+ events
- Co-shard segments + deltas on `cityHash64(segment_id)` (\$15.9)
- `ALTER TABLE ... REPLACE PARTITION` — drop `FINAL` on segments at scale

Analytics & model

- **User-level concurrency** — materialise \$7.4 island-merge → `user_minute_deltas`
- `uniqCombined()` instead of `uniqExact()` in user rollups — HLL at billion-row scale
- Session curve stays exact via ± 1 deltas — never heartbeat-counting
- `concurrency_minute_serving` rollup for hot low-cardinality dims
- **Metadata registry** — central catalog for dynamic `properties.*` (HLD grey box)

Product & integrations

- **Custom MCP server** — force API-parity query templates, not ad-hoc SQL
- LibreChat tool routing → same compiler as `/api/v1/concurrency/chart`
- Richer dashboard — drill-down, alerts, live-event mode (beyond minimal UI)
- Langfuse eval loops — trace quality of MCP-generated analytics answers
- ClickStack SLO dashboards — ingest lag, p95 chart latency, parts read